La gestion du temps en Python

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Différentes façons d'utiliser le temps en Python **4**

A. Le module datetime 4

B. Créer et manipuler des objets datetime 4

C. Exercice : Quiz 7

III - Quelques fonctions avancées **9**

A. Différence entre deux objets datetime 9

B. Utiliser la fonction Sleep() 9

C. Créer un objet datetime depuis une chaîne de caractères 10

D. Exercice : Quiz 11

IV - L'essentiel **13**

V - Auto-évaluation **14**

A. Exercice 14

B. Test 14

Solutions des exercices **16**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

Durée (en minutes) : 60

Environnement de travail : Python 3.12+, PyCharm Community

Pré requis : bases en Python

Contexte

Le temps est une grandeur qu'on est souvent amenés à manipuler. Les exemples sont nombreux : manipuler des données temporelles, réaliser des graphiques, enregistrer les évènements d'une application dans un fichier journal, etc.

Il est important de pouvoir maitriser ce type de donnée, et nous verrons dans ce cours que Python offre énormément de possibilités, en mettant à disposition des fonctionnalités de haut niveau, extrêmement simples à utiliser.

Tous droits réservés à STUDI - Reproduction interdite

II Différentes façons d'utiliser le temps en Python

A. Le module datetime

En python, lorsqu'on est amenés à mettre en œuvre des notions de temps, il existe plusieurs solutions. La plupart du temps, ce sont les modules suivants qui sont utilisés :

- Date
- Datetime

Un objet de type date représente une date calendaire avec une résolution d'une journée,

Un objet de type datetime représente une date avec une résolution bien plus fine, jusqu'à la microseconde.

Le type *datetime*, auquel nous allons particulièrement nous intéresser dans ce cours, contient les attributs suivants :

Attribut	Description
min	Le temps minimum représentable
max	Le temps maximum représentable
resolution	La plus petite unité de temps représentable
year	L'année de la variable
month	Le mois de la variable (1-12)
day	Le jour de la variable
hour	L'heure de la variable courante (0-24)
minute	Les minutes de la variable courante (0-60)
second	Les secondes de la variable courante (0-60)
microsecond	Les microsecondes de la variable courante (0-1 000 000)
tzinfo	Informations sur la zone géographique utilisée
fold	Information sur l'application de l'heure d'été/hiver

B. Créer et manipuler des objets datetime

Commençons par créer un objet de type datetime, et voyons comment récupérer ses différents attributs :

```
1 import datetime
2
3 currentTime = datetime.datetime.now()
4 print(currentTime)
```

Nous venons de créer un objet datetime, initialisé à l'heure courante.

Tous droits réservés à STUDI - Reproduction interdite

Essayons maintenant d'accéder à quelques attributs :

```
1 print(currentTime.year)
2 print(currentTime.minute)
3 print(currentTime.second)
```

Ceci nous renvoie le résultat suivant :

```
> 2024
> 27
> 52
```

Nous sommes à l'année 2024, à la 27e minute de l'heure courante, et à la 52e seconde de la minute courante.

Il est également possible de créer des objets datetime initialisés à des moments spécifiques, voyons ceci dans l'exemple suivant.

Exemple

Supposons qu'on souhaite initialiser un objet datetime au 1er janvier 2025 :

```
1 import datetime
2
3 time1 = datetime.datetime(2025, 1, 1)
4 print(time1)
```

Ceci nous imprime le résultat suivant dans la console :

```
> 2025-01-01 00:00:00
```

Ce qui est bien le résultat attendu.

Comme nous l'avons vu précédemment, une des méthodes couramment utilisée avec les objets datetime est strftime(). Cette fonction permet d'extraire une information spécifique de notre objet datetime pour pouvoir l'afficher sous forme d'une chaine de caractère. Voyons ceci dans un exemple simple.

Exemple

Reprenons notre objet datetime de l'exemple précédent, et supposons que l'on souhaite afficher le mois en toute lettre :

```
1 import datetime
2
3 time1 = datetime.datetime(2025, 1, 1)
4 print(time1)
5
6 month = time1.strftime("%B")
7 print(month)
```

Ceci nous imprime le résultat suivant dans la console :

```
> 2025-01-01 00:00:00
> January
```

On voit que le fait d'avoir spécifié l'argument '%B' dans l'appel à la fonction strftime nous renvoie le mois correspondant à l'objet datetime, écrit en toutes lettres.

Tous droits réservés à STUDI - Reproduction interdite

La fonction strftime permet d'extraire plusieurs informations d'un objet datetime. Les différents arguments utilisables avec cette fonction sont repris dans le tableau suivant :

Argument	Description	Exemple
%a	Jour de la semaine en abrégé	Mon
%A	Jour de la semaine complet	Monday
%w	Jour de la semaine (0-6)	1
%d	Jour du mois (1-31)	10
%b	Nom du mois en abrégé	Jan
%B	Nom complet du mois	January
%m	Mois de l'année (1-12)	1
%y	Année (version courte)	24
%Y	Année (version complète)	2024
%H	Heure (0-23)	12
%I	Heure (0-12)	04
%p	AM/PM	PM
%M	Minute (0-59)	10
%S	Seconde (0-59)	15
%f	Microseconde (0-999999)	15200
%z	Decalage par rapport au temps UTC	+0100
%Z	Zone géographique	CST
%j	Jour de l'année (1-366)	
%U	Semaine de l'année (dimanche = 1e jour de la semaine) (0-53)	15
%W	Semaine de l'année (lundi = 1e jour de la semaine) (0-53)	12
%c	Temps complet au format local	Fri Oct 04 18 :24 :04 2024
%C	Siècle courant	21
%x	Moment de l'année au format local	04/10/2024
%X	Temps au format local	18 :24 :04

Tous droits réservés à STUDI - Reproduction interdite

Le temps « *UTC* » signifie « *Universal Time Coordinated* ». C'est un temps universel et standardisé. En fonction de la zone géographique, on définit des zones horaires spécifiques. Par exemple en France, l'heure est définie en UTC + 2, c'est-à-dire avec 2 heures d'avance par rapport à l'heure UTC. Si l'heure UTC est 08 : 00, il sera 10 : 00 en France.

C. Exercice : Quiz

[solution n°1 p. 16]

Question 1

Quelle est la différence entre un objet « *date* » et un objet « *datetime* » ?

- Ce sont les mêmes objets, il n'y a pratiquement pas de différences
- Seul l'objet « *datetime* » existe en Python
- L'objet « *datetime* » a une résolution plus fine que l'objet « *date* »

Question 2

Que fait la fonction strftime() ?

- Elle remet à zéro la variable de temps
- Elle permet de modifier le temps d'une variable « *datetime* »
- Elle permet de formater une information spécifique contenue dans une variable « *datetime* »

Question 3

Comment initialiser une variable de type datetime à l'heure courante ?

- datetime.current()
- datetime.currentTime()
- datetime.now()

Question 4

Il est possible d'initialiser des objets « *datetime* » à un moment situé dans le futur.

- Vrai
- Faux

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Si je souhaite enregistrer les instants auxquels mon site web est visité, il est préférable d'utiliser des objets de type :

- **A** datetime
- **B** date
- **C** Les deux sont valables

Tous droits réservés à STUDI - Reproduction interdite

III Quelques fonctions avancées

A. Différence entre deux objets datetime

Supposons que nous disposons de deux objets de type datetime.

Dans certains cas, il arrive de vouloir calculer la différence de temps qui sépare deux objets datetime. Nous allons voir que la librairie datetime contient un autre type d'objet bien utile dans ce cas : le timedelta.

Exemple

Reprenons notre objet datetime de l'exemple précédent, et supposons que l'on souhaite afficher le mois en toute lettre :

```
1 from datetime import datetime
2 from datetime import timedelta
3
4 currentTime = datetime.now()
5
6 time1 = datetime(2025, 1, 1)
7
8 timeDiff = time1 - currentTime
9
10 print(timeDiff)
11 print(type(timeDiff))
```

Ceci nous imprime le résultat suivant dans la console :

```
86 days, 22:14:18.544807
<class 'datetime.timedelta'>
```

Comme nous le voyons, le résultat de la soustraction est un objet de type 'timedelta', il représente donc tout simplement le temps qui sépare nos deux objets currentTime et time1, qui sont tous les deux des objets datetime.

B. Utiliser la fonction Sleep()

Dans certaines applications, il est nécessaire d'attendre. Par exemple, parce qu'on doit attendre qu'une action soit terminée avant de continuer, ou parce qu'on souhaite tout simplement décaler certaines actions dans le temps. Nous allons voir un exemple qui utilise la fonction sleep().

Remarque

La fonction sleep() est relative à la librairie native time. Il n'y a donc pas besoin d'importer datetime pour utiliser cette fonction. Il faut cependant importer time.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Reprenons notre objet datetime de l'exemple précédent, et supposons que l'on souhaite afficher le mois en toute lettre :

```
1 from datetime import datetime
2 import time
3
4 time1 = datetime.now()
5 time.sleep(5)
6 time2 = datetime.now()
7
8 timeDiff = time2 - time1
9
10 print(timeDiff)
```

Ceci nous imprime le résultat suivant dans la console :

```
0:00:05.002435
```

Nous avons donc bien deux objets datetime, séparés de 5 secondes.

Remarque

Dans une application, bien que l'appel à la fonction sleep() soit tentant, en général, cela n'est recommandé que pour du débogage ou de manière temporaire. En effet, en général, il est possible de s'en sortir autrement, sans devoir bloquer le programme.

Imaginez que vous développez un serveur Web. Si votre serveur est bloqué dans un sleep(), et qu'une demande extérieure lui parvient, il ne sera pas en mesure de la traiter avant d'avoir fini l'exécution de son sleep().

C. Créer un objet datetime depuis une chaîne de caractères

Il est parfois nécessaire d'interpréter le contenu d'une chaine de caractères (qui est par exemple issue d'un fichier texte préalablement traité), pour en créer un objet datetime facilement manipulable et exploitable.

La librairie datetime contient une fonction particulière qui permet cela. Il s'agit de la fonction strptime() « *String Parse Time* ». Voyons comment elle s'utilise dans un exemple concret.

Exemple

Supposons qu'on dispose d'une chaine de caractères avec l'heure et la date du jour. Nous allons convertir cette chaine de caractère en objet datetime grâce a la fonction strptime() :

```
1 from datetime import datetime
2
3 strTime = '18h54 04/10/2024'
4
5 dt = datetime.strptime(strTime, '%Hh%M %d/%m/%Y')
6
7 print(dt)
```

Ceci nous imprime le résultat suivant dans la console :

```
2024-10-04 18:54:00
```


Tous droits réservés à STUDI - Reproduction interdite

Nous avons donc bien converti notre chaîne de caractères en objet datetime.

D. Exercice : Quiz

[solution n°2 p. 17]

Question 1

Quel est le type d'objet obtenu lors de la soustraction de deux objets datetime ?

- timedelta
- datetime
- date

Question 2

La fonction sleep() fait partie du module datetime :

- Vrai
- Faux

Question 3

À quoi sert la fonction sleep() ?

- A bloquer l'exécution du programme pendant un temps donné
- A continuer l'exécution du programme et revenir à l'endroit du sleep() après un temps donné
- A arrêter l'exécution du programme après un temps donné

Question 4

À quoi sert la fonction strptime() du module datetime ?

- A convertir une chaine de caractères en objet datetime
- A extraire une information spécifique depuis un objet datetime
- Cette fonction n'existe pas dans le module datetime

Question 5

Quel élément d'heure/date correspond au format « *%H* » ?

- Les minutes

Tous droits réservés à STUDI - Reproduction interdite

Les heures au format 0-23 h

Les heures au format 0-12 h

Tous droits réservés à STUDI - Reproduction interdite

IV L'essentiel

Nous avons vu qu'il y avait plusieurs possibilités pour manipuler le temps en Python : les modules time, date et datetime.

Comme toujours en Python, leur utilisation est extrêmement simple. En connaissant seulement quelques fonctions, il est possible de manipuler ces objets efficacement.

Nous avons vu comment créer des objets datetime, en les initialisant de plusieurs façons (temps courant ou temps spécifique). Nous avons vu comment calculer la différence entre deux objets datetime.

Nous avons également vu quelques notions plus avancées, comme la fonction sleep(), qui fait partie du module time, ou encore la fonction strptime(), qui permet de convertir des chaines de caractère en objets datetime.

Tous droits réservés à STUDI - Reproduction interdite

V Auto-évaluation

A. Exercice

Question

[solution n°3 p. 19]

Les lignes d'un fichier log contiennent généralement le temps (heure/minute/seconde) qui correspond à chaque ligne écrite dans ce fichier. Proposez une implémentation d'une fonction « *log(message)* » qui permettra d'imprimer le contenu du message précédé du temps.

B. Test

Exercice 1 : Quiz

[solution n°4 p. 20]

Question 1

Il existe plusieurs modules qui permettent de manipuler le temps en Python :

- **A** datetime
- **B** time
- **C** timeModule

Question 2

Dans l'exemple vu durant ce cours, dont le code et le résultat sont rappelés ci-dessous, pourquoi le résultat n'affiche-t-il pas 5 secondes mais 5,002 secondes ?

```
1 from datetime import datetime
2 import time
3
4 time1 = datetime.now()
5 time.sleep(5)
6 time2 = datetime.now()
7
8 timeDiff = time2 - time1
9
10 print(timeDiff)
```

Ceci nous imprime le résultat suivant dans la console :

```
0:00:05.002435
```

- **A** Car le module datetime n'est pas très précis
- **B** C'est une erreur d'arrondit
- **C** Car les instructions prennent un certain temps à s'exécuter

Question 3

Que fait le code suivant ?

```
1 timeVar.strftime("%H :%M")
```


Tous droits réservés à STUDI - Reproduction interdite

Il imprime l'heure et les minutes contenues dans la variable timeVar

Il écrit l'heure et la minute courante dans la varuable timeVar

Ce code génère une erreur

Question 4

Pourquoi l'utilisation de sleep() doit-il être limité ?

Car le programme est complètement bloqué et indisponible pendant ce temps

Il n'y a aucune raison d'éviter le recours à sleep()

Car c'est dangereux et cela peut faire planter le programme

Question 5

Que signifie l'acronyme UTC ?

Universal Time Coordinated

Unique Time Centralized

Universal Time Center

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 7]

Question 1

Quelle est la différence entre un objet « *date* » et un objet « *datetime* » ?

- Ce sont les mêmes objets, il n'y a pratiquement pas de différences
- Seul l'objet « *datetime* » existe en Python
- ✓ L'objet « *datetime* » a une résolution plus fine que l'objet « *date* »

L'objet « *datetime* » a une résolution temporelle d'une microseconde, alors que l'objet « *date* » s'arrête au jour.

Question 2

Que fait la fonction strftime() ?

- Elle remet à zéro la variable de temps
- Elle permet de modifier le temps d'une variable « *datetime* »
- ✓ Elle permet de formater une information spécifique contenue dans une variable « *datetime* »

La fonction « *strftime()* » permet d'extraire une information spécifique dans une variable « *datetime* », par exemple le nombre de secondes, ou bien le mois. Elle renvoie une chaine de caractères contenant l'information demandée.

Question 3

Comment initialiser une variable de type datetime à l'heure courante ?

- datetime.current()
- datetime.currentTime()
- ✓ datetime.now()

Tous droits réservés à STUDI - Reproduction interdite

datetime.now() permet d'initialiser un objet datetime à l'heure courante (heure actuelle).

Question 4

Il est possible d'initialiser des objets « *datetime* » à un moment situé dans le futur.

- ✓ Vrai
- Faux

C'est tout à fait possible, et c'est d'ailleurs ce que nous avons fait dans l'exemple « *initialiser un objet datetime* ».

Question 5

Si je souhaite enregistrer les instants auxquels mon site web est visité, il est préférable d'utiliser des objets de type :

- **A** datetime
- **B** date
- **C** Les deux sont valables

Bien que les deux soient valables, étant donné qu'ils n'ont pas la même résolution temporelle, l'objectif ne sera pas le même. Si je peux me satisfaire du jour de visite, je peux utiliser un objet « *date* ». Sinon, j'utiliserai un objet « *datetime* », beaucoup plus précis.

Solution n°2

[exercice p. 11]

Question 1

Quel est le type d'objet obtenu lors de la soustraction de deux objets datetime ?

- ✓ timedelta
- datetime
- date

Tous droits réservés à STUDI - Reproduction interdite

L'objet obtenu est de type timedelta. C'est un objet qui contiendra le temps entre les deux objets datetime utilisés dans la soustraction.

Question 2

La fonction sleep() fait partie du module datetime :

- Vrai
- ✓ Faux

La fonction sleep() fait partie du module time, et non du module datetime.

Question 3

À quoi sert la fonction sleep() ?

- ✓ A bloquer l'exécution du programme pendant un temps donné
- A continuer l'exécution du programme et revenir à l'endroit du sleep() après un temps donné
- A arrêter l'exécution du programme après un temps donné

La fonction sleep() est ce qu'on appelle une fonction bloquante. L'appel à sleep() suspendra l'exécution du programme pendant un temps donné. Ce temps est spécifié dans l'argument de sleep() et est exprimé en secondes.

Question 4

À quoi sert la fonction strptime() du module datetime ?

- ✓ A convertir une chaine de caractères en objet datetime
- A extraire une information spécifique depuis un objet datetime
- Cette fonction n'existe pas dans le module datetime

Cette fonction sert à convertir une chaîne de caractères en objet datetime. La deuxième proposition correspond à la fonction strftime().

Question 5

Quel élément d'heure/date correspond au format « *%H* » ?

Tous droits réservés à STUDI - Reproduction interdite

Les minutes

✓ Les heures au format 0-23 h

Les heures au format 0-12 h

Le format %H correspond à l'heure au format 24 h. Pour l'heure au format 0-12 h, il aurait fallu utiliser %h et non %H. Les minutes sont spécifiées par « *%M* ».

[exercice p. 14]

Solution n°3

Dans cet exercice, nous allons d'abord définir deux fonctions :

- trace(), qui permet de « *tracer* » ou d'enregistrer une ligne de log. Cette fonction doit d'abord récupérer le temps exact à laquelle elle est appelée, pour ensuite imprimer le message souhaité précédé du temps,
- get_time() permet tout simplement de renvoyer le temps courant

Ensuite, nous définissons un programme qui réalise 10 boucles. Pour simuler un process, nous avons utilisé l'instruction time.sleep(1), qui bloquera le programme durant 1 seconde à chaque appel.

Le code suivant répond au besoin :

```
1 from datetime import datetime
2 import time
3
4 def trace(myLogLine):
5     timestamp = get_time()
6     print(f"{timestamp} / {myLogLine}")
7
8
9 def get_time():
10     return datetime.now()
11
12
13 for i in range(10):
14     time.sleep(1)
15     trace("process step :" + str(i))
```

Ce programme devrait vous donner le résultat suivant :

```
2024-10-13 10:29:46.717466 / process step :0
2024-10-13 10:29:47.717992 / process step :1
2024-10-13 10:29:48.718487 / process step :2
2024-10-13 10:29:49.718659 / process step :3
2024-10-13 10:29:50.718779 / process step :4
2024-10-13 10:29:51.719105 / process step :5
2024-10-13 10:29:52.719464 / process step :6
2024-10-13 10:29:53.720122 / process step :7
2024-10-13 10:29:54.720616 / process step :8
```

Tous droits réservés à STUDI - Reproduction interdite

```
2024-10-13 10:29:55.721060 / process step :9
```

On constate que chaque ligne est bien appelée avec un intervalle d'une seconde, ce qui est attendu étant donné que la seule instruction de notre programme est time.sleep(1).

Solution n°4

[exercice p. 14]

Question 1

Il existe plusieurs modules qui permettent de manipuler le temps en Python :

- **A** datetime
- **B** time
- **C** timeModule

Datetime et time sont deux modules Python qui permettent d'exploiter le temps. timeModule n'existe pas.

Question 2

Dans l'exemple vu durant ce cours, dont le code et le résultat sont rappelés ci-dessous, pourquoi le résultat n'affiche-t-il pas 5 secondes mais 5,002 secondes ?

```python
from datetime import datetime
import time

time1 = datetime.now()
time.sleep(5)
time2 = datetime.now()

timeDiff = time2 - time1

print(timeDiff)
```

Ceci nous imprime le résultat suivant dans la console :

```
0:00:05.002435
```

- **A** Car le module datetime n'est pas très précis
- **B** C'est une erreur d'arrondit
- **C** Car les instructions prennent un certain temps à s'exécuter

Les instructions Python peuvent prendre un certain temps pour s'exécuter. En plus d'être légèrement supérieur à 5 secondes (dans ce cas, 2,4 ms en plus), chaque exécution du code donnera un résultat légèrement différent.

Tous droits réservés à STUDI - Reproduction interdite

Question 3

Que fait le code suivant ?

```
1 timeVar.strftime("%H :%M")
```

- ✓ Il imprime l'heure et les minutes contenues dans la variable timeVar
- Il écrit l'heure et la minute courante dans la varuable timeVar
- Ce code génère une erreur

Ce code imprime l'heure et les minutes contenues dans la variable timeVar dans la console.

Question 4

Pourquoi l'utilisation de sleep() doit-il être limité ?

- ✓ Car le programme est complètement bloqué et indisponible pendant ce temps
- Il n'y a aucune raison d'éviter le recours à sleep()
- Car c'est dangereux et cela peut faire planter le programme

L'utilisation de sleep() rend indisponible l'application pendant un certain temps. C'est pour cette raison que cela est réservé à des cas bien spécifiques.

Question 5

Que signifie l'acronyme UTC ?

- ✓ Universal Time Coordinated
- Unique Time Centralized
- Universal Time Center

UTC signifie Universal Coordinated Time. Il s'agit d'un standard universel de définition du temps.

Tous droits réservés à STUDI - Reproduction interdite